

I2204

Imperative programming II

Outline

- Recursivity
- Structures
- Pointers and arrays
- Lists
- Files
- Creation of complete modules

Chapter 1

Recursion

Outline

- Introduction
- Example: factorial function
- Analysis of the recursive method
- Classical problems
- Exercises

Introduction

- An algorithm uses one of the two methods
 - iterative method, called also **iterative function**
 - based on loops and terminates its execution when we exit the loop
 - recursive method, called also **recursive function**
 - this is when a function calls itself
 - one or many time to accomplish a given task
- Motivation
 - recursive functions **simplify** the implementation of algorithms
 - they give an elegant and comprehensive representation
 - iterative functions are more efficient in terms of **performance**
- A recursive function can always be implemented in an iterative form

Example: factorial function

- Write an iterative and recursive function to calculate the factorial of a number
- **Iterative notation**

$$\begin{aligned}0! &= 1 \\1! &= 1 \\2! &= 1 \times 2 \\3! &= 1 \times 2 \times 3\end{aligned}$$

$$\begin{aligned}\text{General form:} \\n! &= 1 \text{ si } n = 0 \\n! &= 1 * 2 * \dots * (n-1) * n \text{ if } n > 0\end{aligned}$$

- **Recursive notation**

$$\begin{aligned}4! &= 4 \times 3 \times 2 \times 1 \\&= 4 \times (3 \times 2 \times 1) \\&= 4 \times 3!\end{aligned}$$

$$\begin{aligned}\text{General form:} \\n! &= 1 \text{ si } n = 0 \\n! &= n * (n-1)! \text{ if } n > 0\end{aligned}$$

$n!$ is defined in terms of itself

Given $f(n)=n!$ and $f(n-1)=(n-1)!$ Then $f(n)=n*f(n-1)$

Example: factorial function

Iterative and Recursive

```
#include <iostream>
Using namespace std;
double factorialIter (int n) { //iterative function
    double facto=1;
    for(int i=1;i<=n;i++)
        facto=i*facto;
    return facto;
}
double factorial(int n){ //recursive function
    if (n <= 1)          // cas de base
        return 1;
    return n*factorial (n-1); // cas récuratif
}
int main(){
    int n;
    for(int n=0; n<=10; n++)
        cout<<n<<"! = "<<factorial(n)<<endl;
    return 0;
}
```

Output

0!	=	1
1!	=	1
2!	=	2
3!	=	6
4!	=	24
5!	=	120
6!	=	720
7!	=	5040
8!	=	40320
9!	=	362880
10!	=	3628800

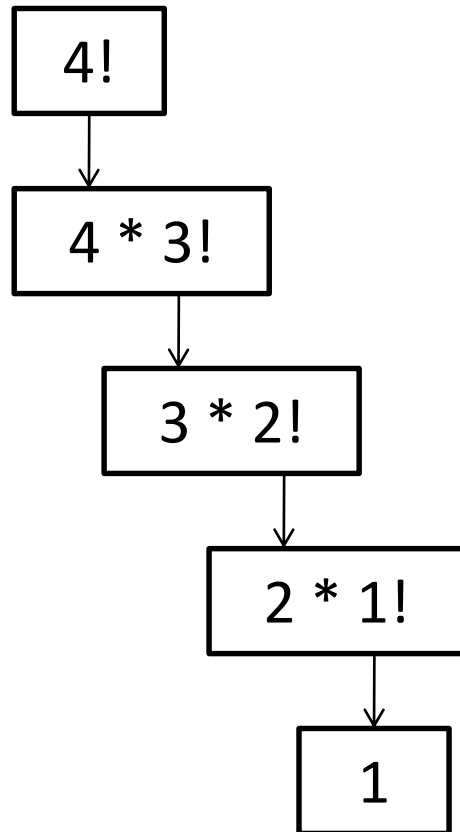
Example: factorial function

Recursive call representation

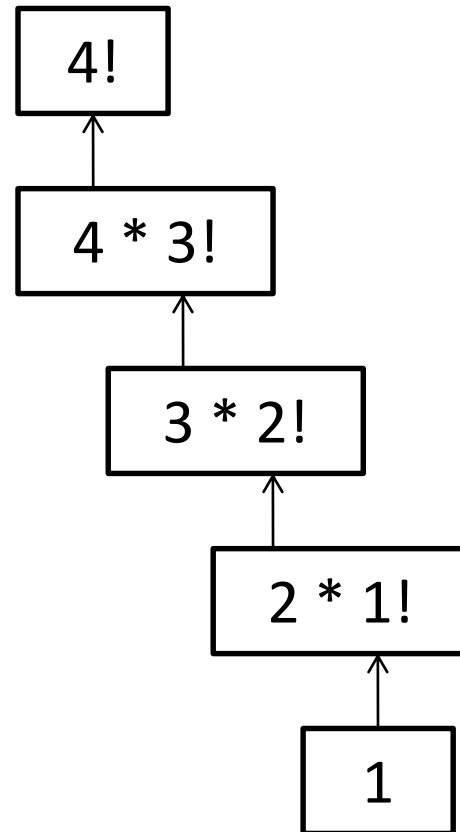
Recursive call			
1	2	3	4
factorial (4)	factorial(3)	factorial(2)	factorial(1)
n=4 4*factorial(3); $\longrightarrow$ $\longleftarrow$ 24 return 4*6;	n = 3; 3*factorial (2); $\longrightarrow$ $\longleftarrow$ 6 return 3*2;	n = 2; 2*factorial (1); $\longrightarrow$ $\longleftarrow$ 2 return 2*1;	n = 1; $\longleftarrow$ 1 return 1;

Example: factorial function

Recursive call representation (cont.)



Recursive call process



Values returned in each recursive call

final value = 24

Returns $4! = 4 * 6 = 24$

Returns $3! = 3 * 2 = 6$

Returns $2! = 2 * 1 = 2$

Returns 1

Example: factorial function

Analysis

- The recursive factorial function is based on:
 - a **basic case** that is a termination condition, i.e. a condition that terminates the recursive call
 - e.g., for $n \leq 1$, the answer is known and equal 1
 - a **general case** that is a general expression for calculating the factorial function in terms of itself
 - e.g., $f(n) = n * f(n-1)$ that is $n! = n * (n-1)!$

Analysis of the recursive method

- To construct a recursive solution, we should
 - understand the problem and find the **parameters** of the recursive function
 - find the **basic case(s)**
 - for which the solution(s) is known
 - which is a termination condition of the recursion
 - this will prevent infinity recursion
 - find the **general case**
 - where the solution can be expressed in terms of (part of) itself
 - which is a general expression

CLASSICAL PROBLEMS

Power function

- Write an iterative and recursive function that calculates x^n for $n > 0$
- Iterative notation
 $x^n = x * x * \dots x * x$; n multiplication by x
- Recursive notation
 - basic case: $x^1 = x$
 - general case: $x^n = x * x^{n-1}$
- If $n \geq 0$, the basic case is $x^0 = 1$

Power function

Iterative and recursive

```
double powerIter(int x, int n) {
    double pow=1;
    for(int i=1;i<=n;i++)
        pow=x*pow;
    return pow;
}
//x^n=x*x^(n-1)
double power(int x, int n){
    if (n == 0)
        return 1;
    return x*power(x,n-1);
}
```

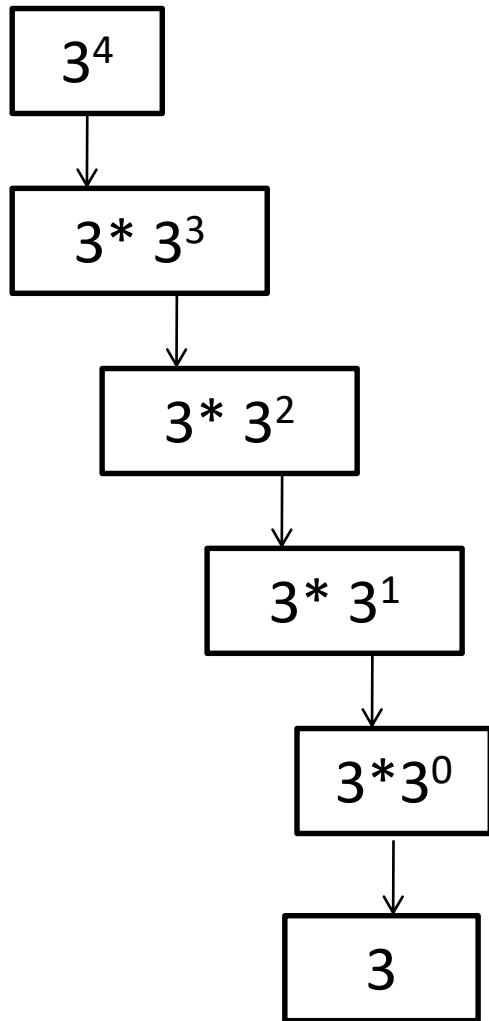
Power function

Recursive calls representation

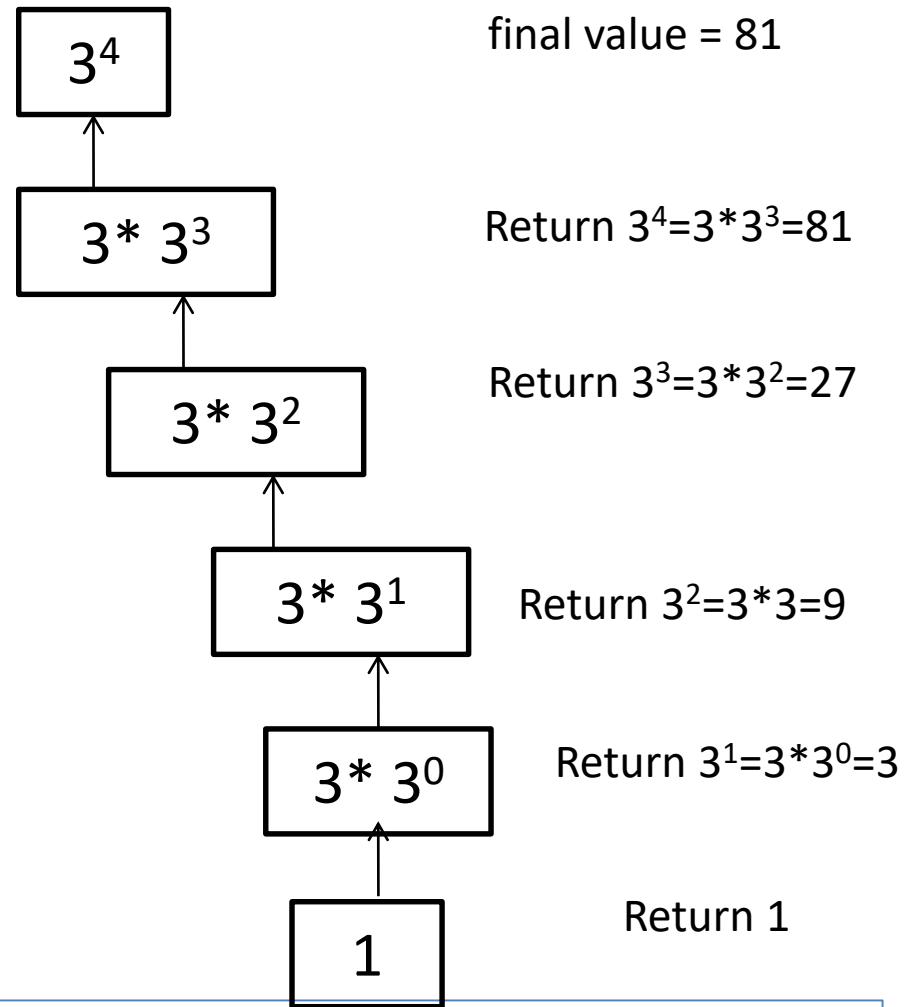
Appels rékursifs				
1	2	3	4	5
power(3,4)	power(3,3)	power(3,2)	power(3,1)	Power(3,0)
x=3; n=4; 3*power (3,3); ← 81 return 3*27;	x=3; n=3; 3*power (3,2); 27 return 3*9;	x=3; n=2; 3*power (3,1); 9 return 3*3;	x=3; n=1; 3 return 3;	X=3;n=0 Return 1

Power function

Recursive calls representation (cont.)



Recursive call process



Returned values to each recursive call 16

Power function

2nd method, more efficient

- If n is even: $x^n = (x^{n/2})^2$; e.g.: $3^4 = (3^2)^2$
- If n is odd: $x^n = (x^{(n-1)/2})^2 * x$; e.g.: $3^5 = (3^2)^2 * 3$
- Then the recursive calls number will be reduced almost to half

```
double power(int x, int n) {  
    double res;  
    if (n == 1)  
        return x;  
    if (n%2==0) {  
        res=power(x,n/2) ;  
        return res*res;  
    }  
    else {  
        res=power(x,n/2) ;  
        return x*res*res;  
    }  
}
```

Binary search function

- It is also called dichotomic search
 - because it divides the array elements in two different sub-parts
 - and repeat the division until we find the index of the searched element
 - in the middle of the array sub-part
- Here, the array should be sorted

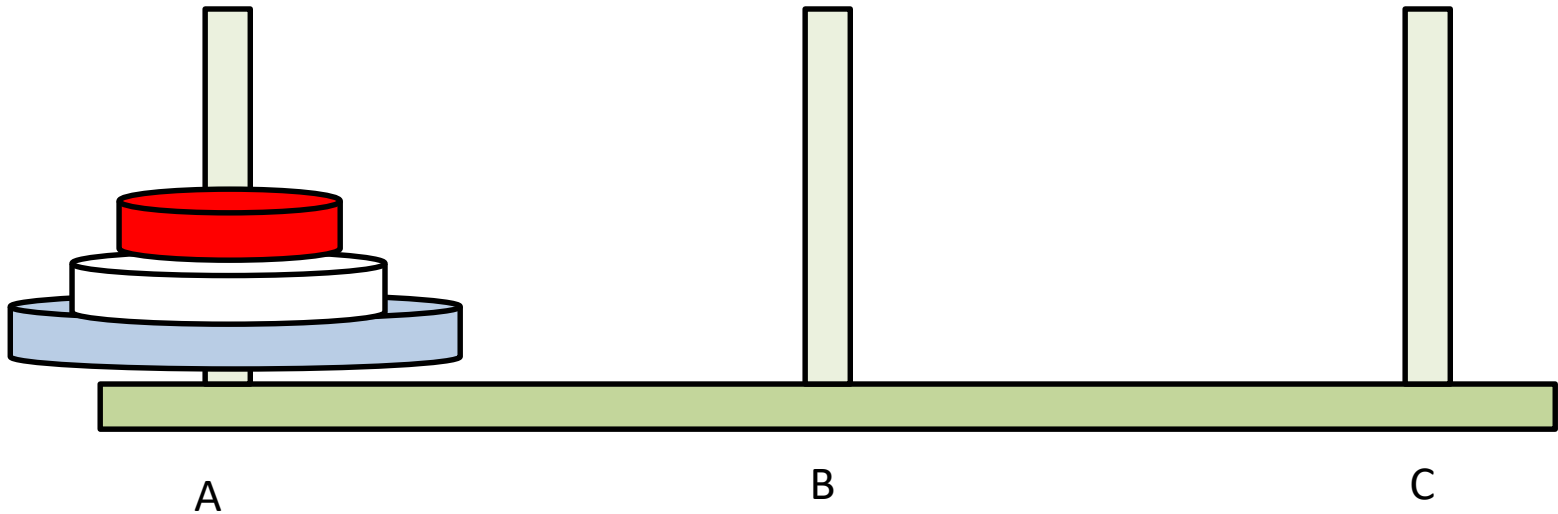
Binary search function

```
//low means small indices
int BinSearch(int tab[], int low, int high, int x){
    int mid;
    if(low>high){
        return -1;
    }
    mid=(low+high)/2;
    if(x==tab[mid])
        return mid;
    if(x<tab[mid])
        return BinSearch(tab,low, mid-1,x);
    else
        return BinSearch(tab,mid+1,high,x);
}
```

General case: divide the array in two parts at each recursive call

Basic case: stop when x is found in the middle of a sub-part

Towers of Hanoi



- Three pegs: A, B and C
- n disks with different unique diameter are placed initially as shown in the figure
- The problem consists in moving n disks in another peg by respecting the following conditions
 - move one disk in a time
 - never put in a small disk a larger disk
 - use an intermediate peg

Towers of Hanoi (cont.)

- Parameters: n and A (starting peg), B (arriving peg) and C (intermediate peg)
- Basic case:
 - If $n=0$, nothing to do
 - If $n=1$, move a disk from A to B
- General case:
 1. move $n-1$ disks from A to C
 2. move 1 disk from A to B
 3. move $n-1$ disks from C to B
- The algorithm is implemented in a very elegant way with a recursive function.

Towers of Hanoi (cont.)

```
void hanoi(int n, char source, char dest, char interm){  
    if(n==1)  
        cout<<"Move the disk 1 from "<< source  
            <<" to " <<dest <<endl;  
    else{  
        Hanoi(n-1, source,interm,dest)    ;  
        cout<<"Move the disk "<< n << " from "  
            <<source<<" to "<<dest<<endl;  
        Hanoi(n-1,interm,dest,source) ;  
    }  
}
```

Exemple, move 3 disks from 'A' to 'B'

Calling **Hanoi(3,'A', 'B', 'C')** generates the following output

```
Move the disk 1 from A to B  
Move the disk 2 from A to C  
Move the disk 1 from B to C  
Move the disk 3 from A to B  
Move the disk 1 from C to A  
Move the disk 2 from C to B  
Move the disk 1 from A to B
```

EXERCISES

Exercise 1: Write an iterative and recursive functions that calculates

$S_n = \text{tab}[0] + \text{tab}[1] + \dots + \text{tab}[n-1]$ where n is the elements number

Exercise 2: Write a function that displays the binary value of a decimal number n

Exercise 3: Write a recursive function that displays an array of n characters

Exercise 4: Write a recursive function that displays the reverse of an array of **n** characters

Exercise 5: Write a recursive function that determines the maximum of elements in a array **tab** of **n** integers

Exercise 6: Write a recursive function that computes the **n**th element of the Fibonacci series

$$\text{Fib}(n) = n \text{ if } n \leq 1$$

$$\text{Fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$$